

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA1504
---------------------	---	---------------------	---------

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 40

Duration :60 Minutes

Code of Conduct

I S Jefna Princy (192572058) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Assessment Tasks

Industry Problem Solving Task

1. **Retail Data Optimization Problem** - A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.

Retail Data Optimization Problem

Problem Understanding

A large retail chain operates both physical stores and an online platform. It generates huge amounts of data from sales transactions, customer profiles, inventory systems, websites, mobile applications, and customer interactions. The data is generated continuously and exists in structured as well as unstructured forms.

The major challenges are handling **high data volume, high data velocity, data variety, data integration, data quality, and customer data security**. The organization needs a Big Data solution

that can provide real-time inventory visibility, accurate demand forecasting, customer behaviour analysis, and personalized marketing recommendations.

Proposed Big Data Architecture

A scalable cloud-based Big Data architecture can be designed using the following layers:

Data Sources → Data Ingestion → Data Storage → Data Processing → Analytics & Machine Learning → Visualization & Applications

Data Sources

Data is collected from **point-of-sale systems, e-commerce websites, mobile applications, inventory systems, customer reviews, loyalty programs, and social media.**

Data Ingestion

Apache Kafka can collect real-time sales and customer activity data. Batch data from existing databases can be transferred using cloud-based data ingestion services.

Data Storage

A cloud-based **Data Lake** such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage can store structured, semi-structured, and unstructured data. A **Data Warehouse** can store processed structured data for reporting and business intelligence.

Data Processing

Apache Spark can process large volumes of retail data using distributed computing. **Spark Streaming** can process real-time sales and inventory events, while batch processing can be used for historical analysis.

Analytics and Machine Learning

Machine learning models can be used for:

- **Demand forecasting** to predict future product requirements.
- **Customer segmentation** to group customers according to purchasing behaviour.
- **Recommendation systems** to suggest products based on customer preferences.
- **Inventory optimization** to identify products that need replenishment.
- **Sales trend analysis** to identify seasonal and changing customer demands.

Architecture Flow

Physical Stores + Website + Mobile App + Inventory + Customer Data



Kafka / Cloud Data Ingestion



Cloud Data Lake



Apache Spark Processing



Data Warehouse + Machine Learning Models



BI Dashboards + Recommendation System + Inventory Management

Demand Forecasting

Demand forecasting models can analyze **historical sales, seasonal trends, holidays, promotions, product popularity, and customer behaviour** to predict future demand.

For example, if historical data shows that a particular product has high demand during a festival season, the system can predict increased demand and automatically recommend increasing its stock before the festival.

Personalized Marketing

Customer purchase history and browsing behaviour can be analyzed to create personalized recommendations. **Collaborative filtering** can recommend products based on similar customers, while **content-based filtering** can recommend products based on previously viewed or purchased items.

This improves customer engagement and can increase sales.

Security and Risk Management

Customer information must be protected using **encryption at rest and in transit**. **Role-Based Access Control (RBAC)** should ensure that employees can access only the information required for their roles.

Sensitive customer information should be **masked or anonymized** before being used for analytics. Strong authentication, audit logs, continuous monitoring, secure backups, and disaster recovery mechanisms should also be implemented.

Data quality checks should be performed to identify **missing, duplicate, inconsistent, or incorrect data** before analytics.

Scalability and Reliability

The cloud architecture can automatically scale computing and storage resources according to the amount of incoming retail data. Distributed processing using Apache Spark prevents a single machine from becoming a bottleneck.

Replication, backup, fault tolerance, and disaster recovery mechanisms ensure that business operations continue even if individual systems fail.

Expected Benefits

The proposed Big Data solution provides **real-time inventory visibility, improved demand forecasting, reduced stock-outs and overstocking, personalized customer recommendations, faster decision-making, and improved operational efficiency.**

The cloud-based architecture is scalable and can support the retail chain as the number of stores, customers, transactions, and online activities increases.

Conclusion

A cloud-based Big Data architecture combining **Kafka, cloud data lakes, Apache Spark, data warehouses, machine learning, and real-time analytics** provides an effective solution for retail data optimization. It transforms large amounts of raw retail data into useful business insights while maintaining scalability, security, reliability, and efficient decision-making.

2. **Social Media Fraud Detection Problem** - A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.

Social Media Fraud Detection Problem

Problem Understanding

A social media platform processes billions of user interactions every day, including **posts, comments, likes, shares, messages, logins, and account activities**. The platform needs to detect fake accounts, spam, automated bots, suspicious behaviour, and coordinated fraudulent activities in real time.

The main challenges are **high data volume, high data velocity, real-time processing, changing fraud patterns, false positives, and protection of user privacy**. Therefore, a scalable Big Data architecture with stream processing and machine learning is required.

Proposed Big Data Architecture

The proposed system follows this architecture:

User Activities → Data Ingestion → Stream Processing → Feature Extraction → Fraud Detection → Risk Score → Action

Data Sources

The system collects data from **posts, comments, likes, shares, login attempts, account creation, IP addresses, device information, messages, and user interaction patterns**.

Data Ingestion

Apache Kafka can be used to collect billions of real-time events from different sources. Kafka provides high-throughput and fault-tolerant event streaming.

Stream Processing

Apache Flink or Apache Spark Streaming can process incoming events continuously with low latency. The system can calculate user behaviour features immediately instead of waiting for batch processing.

Feature Extraction

Important features for fraud detection include:

- Frequency of posts and comments
- Number of login attempts
- Sudden changes in IP addresses
- Device changes
- Account age
- Follower-to-following ratio
- Repeated or identical content
- Unusual activity timings
- Rapid likes, shares, or follows
- Connections with already detected fraudulent accounts

Fraud Detection Using Machine Learning

Machine learning can classify accounts and activities as **normal or suspicious**.

Supervised learning algorithms such as Random Forest and XGBoost can detect previously known fraud patterns.

Anomaly detection algorithms such as Isolation Forest can identify unusual behaviour that differs significantly from normal users.

A hybrid approach combining supervised learning and anomaly detection provides better protection because it can detect both known and new fraud patterns.

Real-Time Processing Flow

User Interaction

↓

Apache Kafka

↓

Apache Flink / Spark Streaming

↓

Feature Extraction



Machine Learning Fraud Detection Model



Risk Score Generation

Low Risk → Allow Activity

Medium Risk → Additional Verification

High Risk → Block / Restrict Account + Generate Alert

Graph-Based Fraud Detection

Graph analytics can also be used to identify coordinated fraudulent accounts. Users can be represented as nodes, while relationships such as follows, messages, likes, shared devices, and common IP addresses can be represented as connections.

If a large group of accounts shows highly coordinated behaviour or shares common devices and IP addresses, the system can assign a higher fraud risk score.

Security and Privacy

User data must be protected using **encryption during storage and transmission**. **Role-Based Access Control** should restrict access to authorized employees.

Personally identifiable information should be **anonymized or pseudonymized** wherever possible. Strong authentication, audit logging, secure APIs, network security, and continuous monitoring should be implemented.

The system should also follow **data minimization**, collecting only the information required for fraud detection.

Accuracy and False Positive Management

Fraud detection systems must avoid incorrectly blocking genuine users. Therefore, activities with uncertain risk scores can be sent for **additional verification or human review** rather than immediately blocking the account.

The machine learning model should be continuously monitored and retrained using newly confirmed fraud cases so that it can adapt to changing attack patterns.

Scalability and Reliability

The cloud-based architecture can automatically scale according to the number of incoming social media events. Kafka provides distributed event processing, while Flink or Spark can process data across multiple nodes.

Data replication, fault tolerance, backups, and disaster recovery mechanisms should be implemented to ensure continuous availability.

Expected Benefits

The proposed Big Data solution provides **real-time fraud detection, faster identification of fake accounts and bots, reduced spam, improved platform security, scalable processing, and better protection of genuine users.**

The combination of **Kafka, stream processing, machine learning, anomaly detection, and graph analytics** enables the platform to detect both known and previously unseen fraudulent behaviour.

Conclusion

A real-time Big Data fraud detection system is essential for a large social media platform because fraudulent activities can occur continuously at massive scale. By combining **Apache Kafka, Apache Flink or Spark Streaming, machine learning, anomaly detection, graph analytics, and cloud computing**, the platform can detect suspicious activities quickly and take appropriate action while maintaining security, privacy, scalability, and reliability.

3. **Government Data Platform Problem** A government agency is building a national data platform. It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.

Government Data Platform Problem

Problem Understanding

A government agency is building a national data platform to integrate information from multiple departments such as **healthcare, education, transportation, taxation, public welfare, and citizen services.**

The major challenge is that different departments may use different databases, data formats, applications, and security policies. The platform must provide **secure data sharing, interoperability, scalability, data quality, privacy, and effective governance.**

A cloud-based Big Data platform can integrate these distributed data sources and provide a common environment for storage, processing, analytics, and decision-making.

Proposed Big Data Architecture

The proposed architecture follows:

Departmental Systems → Secure Data Ingestion → Data Lake → Data Processing & Integration → Data Warehouse → Analytics & Citizen Services

Data Sources

Data can be collected from:

- Healthcare databases

- Education systems
- Taxation systems
- Transportation systems
- Welfare schemes
- Citizen service applications
- Government documents
- IoT and sensor systems

These sources can generate structured, semi-structured, and unstructured data.

Data Ingestion

Secure **APIs, message queues, batch transfers, and streaming platforms** can be used to collect data from different government departments.

Apache Kafka can be used where real-time event streaming is required. Secure APIs can allow authorized departments to exchange information without directly exposing internal databases.

Data Storage

A centralized **cloud data lake** can store large volumes of raw government data in its original format.

A data warehouse can store cleaned and standardized data for reporting and analytics.

Cloud storage provides scalability, allowing new departments and datasets to be added without major changes to the infrastructure.

Data Processing

Apache Spark can process and transform large datasets from different departments.

The processing layer performs:

- Data cleaning
- Data transformation
- Data integration
- Duplicate removal
- Data validation
- Data standardization
- Data quality checking

Data Integration Flow

Healthcare + Education + Taxation + Transport + Welfare Data



Secure APIs / Kafka / Data Ingestion



Central Cloud Data Lake



Data Cleaning + Transformation + Integration



Governed Data Warehouse



Analytics + Dashboards + Citizen Services

Data Governance Strategy

A strong data governance framework is essential because government data contains sensitive citizen information.

A **Data Governance Committee** should define policies for data ownership, access, sharing, retention, quality, and security.

A common **data dictionary** should be maintained so that different departments use consistent definitions, formats, and standards.

Metadata management should record information about the source, meaning, owner, quality, and usage of every important dataset.

Regular data quality checks should identify **missing, duplicate, inconsistent, and inaccurate records**.

Security and Privacy

Government data requires strong security because it may contain sensitive citizen information.

Role-Based Access Control (RBAC) should ensure that users can access only the information required for their responsibilities.

Multi-Factor Authentication (MFA) should be used for accessing sensitive systems.

Data must be protected using **encryption at rest and encryption in transit**.

Sensitive citizen information should be **anonymized or pseudonymized** before being used for analytics whenever individual identification is not required.

Detailed **audit logs** should record who accessed, modified, or transferred data.

Network segmentation, firewalls, intrusion detection, vulnerability testing, and continuous security monitoring should also be implemented.

Interoperability

Different government departments may use different technologies and data formats. Therefore, the platform should use **standardized APIs, common data formats, metadata standards, and interoperability protocols**.

An API gateway can control and monitor communication between departments while preventing unauthorized access.

This allows departments to exchange information without completely replacing their existing systems.

Scalability and Reliability

The platform should use cloud infrastructure so that storage and computing resources can scale according to data volume.

Distributed processing using Apache Spark allows large datasets to be processed across multiple machines.

Data replication, automated backups, fault tolerance, high availability, and disaster recovery should be implemented to ensure continuous government services.

Expected Benefits

The proposed platform provides **secure data sharing between government departments, improved interoperability, better data quality, faster decision-making, reduced duplicate data collection, improved citizen services, and scalable national-level analytics.**

It also creates a common platform where government agencies can analyze data while maintaining strict security and governance controls.

Conclusion

A centralized cloud-based Big Data platform can effectively solve the challenges of integrating large-scale government data. By combining **cloud storage, secure APIs, Apache Kafka, Apache Spark, data governance, encryption, access control, metadata management, and analytics**, government departments can securely share and analyze information.

The proposed solution is **scalable, secure, interoperable, reliable, and suitable for national-level deployment**, while protecting citizen privacy and supporting efficient government decision-making.

4. Banking Fraud Detection Problem

A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data.

Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

Banking Fraud Detection Problem

Problem Understanding

A bank processes millions of financial transactions every minute through **ATMs, credit cards, online banking, mobile banking, UPI, and other digital payment systems**. Among these transactions, some may be fraudulent and must be detected immediately to prevent financial loss.

The major challenges are **high transaction volume, high velocity, low-latency processing, changing fraud patterns, high accuracy, scalability, and protection of sensitive financial data.**

Therefore, a real-time Big Data pipeline using **stream processing and machine learning** is required to detect fraudulent transactions quickly and accurately.

Proposed Big Data Architecture

The proposed fraud detection pipeline follows:

Transaction Sources → Data Ingestion → Stream Processing → Feature Engineering → Machine Learning → Risk Score → Decision → Action

Transaction Sources

The system collects transactions from **credit cards, debit cards, ATMs, UPI, mobile banking, online banking, and international payments.**

Additional information such as device details, location, merchant information, transaction time, and customer spending history can also be used for fraud detection.

Real-Time Data Ingestion

Apache Kafka can be used to collect and distribute transaction events in real time.

Kafka provides **high throughput, scalability, fault tolerance, and reliable event streaming**, making it suitable for processing a very large number of banking transactions.

Stream Processing

Apache Flink or Apache Spark Streaming can process transactions continuously with very low latency.

Each transaction can be analyzed immediately instead of waiting for batch processing. This allows the bank to identify suspicious transactions within seconds.

Feature Engineering

Important features can be extracted from every transaction, such as:

- Transaction amount
- Transaction location
- Transaction time
- Customer's normal spending pattern
- Transaction frequency
- Merchant category
- Device information
- IP address
- Previous transaction history

- Sudden change in geographical location

These features are provided to the fraud detection model.

Fraud Detection Using Machine Learning

Machine learning algorithms can classify transactions as **legitimate or fraudulent**.

Random Forest and XGBoost can be used for supervised fraud classification because they can learn patterns from previously identified fraudulent transactions.

Isolation Forest can be used for anomaly detection to identify unusual transactions that do not follow the customer's normal behaviour.

A hybrid approach combining classification and anomaly detection can improve fraud detection accuracy and help identify new fraud patterns.

Fraud Detection Flow

Bank Transaction

↓

Apache Kafka

↓

Apache Flink / Spark Streaming

↓

Feature Extraction

↓

Machine Learning Fraud Detection Model

↓

Risk Score Generation

Low Risk → Approve Transaction

Medium Risk → Additional Authentication / OTP

High Risk → Block Transaction + Generate Fraud Alert

Historical Data Analysis

Historical transaction data can be stored in a **cloud data lake** for long-term analysis.

Apache Spark can process historical data to identify fraud patterns, evaluate model performance, and retrain machine learning models.

This creates a combination of:

Real-Time Processing → Immediate Fraud Detection

Batch Processing → Historical Analysis and Model Improvement

Using both approaches allows the fraud detection system to continuously improve.

Security and Risk Management

Banking data is highly sensitive and must be protected using strong security mechanisms.

Encryption should be used for data at rest and in transit. Sensitive card and account information should be protected using **tokenization and masking**.

Role-Based Access Control should ensure that employees can access only the data required for their job responsibilities.

Multi-Factor Authentication should be used for sensitive systems. Detailed audit logs should record data access and system activities.

Continuous monitoring, secure APIs, firewalls, intrusion detection, regular security testing, backup, and disaster recovery mechanisms should also be implemented.

Low-Latency and Scalability

Low latency is critical in banking because a fraudulent transaction must be identified before it is completed.

Kafka provides high-speed event ingestion, while Flink or Spark Streaming provides distributed stream processing.

Cloud infrastructure can automatically scale computing resources when transaction volume increases, such as during festivals, sales events, or peak banking hours.

High availability and fault tolerance should be implemented so that the fraud detection system continues operating even if individual servers fail.

Expected Benefits

The proposed Big Data pipeline provides:

- **Real-time fraud detection**
- **Low-latency transaction analysis**
- **High scalability**
- **Improved fraud detection accuracy**
- **Reduced financial losses**
- **Continuous monitoring**
- **Protection of customer information**
- **Faster response to suspicious transactions**

The system can also adapt to new fraud patterns by continuously training machine learning models using newly detected and verified fraud cases.

Conclusion

A real-time Big Data fraud detection system is essential for modern banking because millions of transactions are generated continuously. By combining **Apache Kafka, Apache Flink or Spark**

Streaming, machine learning, anomaly detection, cloud computing, and secure data management, fraudulent transactions can be detected quickly and accurately.

The proposed solution provides **low latency, high scalability, strong security, continuous monitoring, and reliable fraud detection**, making it suitable for large-scale banking environments.

5. Agriculture Smart Farming Problem

A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

Agriculture Smart Farming Problem

Problem Understanding

Modern agriculture generates large amounts of data from **IoT sensors, weather stations, drones, satellites, GPS devices, and agricultural equipment**. This data can be used to improve crop production, reduce water consumption, detect diseases, and optimize the use of fertilizers and pesticides.

The major challenges are **large data volume, continuous data generation, integration of different data sources, real-time monitoring, connectivity issues, and secure data management**.

A cloud-based Big Data and IoT solution can collect, process, and analyze this data to provide useful recommendations to farmers.

Proposed Big Data Architecture

The proposed smart farming architecture follows:

Sensors + Drones + Weather Stations + Satellites → IoT Gateway → Data Ingestion → Cloud Data Lake → Data Processing → Machine Learning → Farmer Dashboard /

Automated Actions

Data Sources

IoT sensors can continuously collect:

- Soil moisture
- Soil temperature
- Humidity
- Soil pH

- Nutrient levels
- Rainfall
- Crop conditions

Drones can capture images of agricultural fields, while satellites can provide large-scale information about vegetation and crop health.

Weather stations provide information such as **temperature, rainfall, wind speed, humidity, and weather conditions.**

Data Ingestion

An **IoT Gateway** can collect data from different sensors and send it securely to the cloud.

Apache Kafka can be used for continuous real-time data ingestion. It can handle large numbers of sensor events and distribute them to different processing systems.

Data Storage

A cloud-based **Data Lake** can store sensor readings, drone images, satellite images, weather information, and historical farming data.

The data lake can store structured, semi-structured, and unstructured data, making it suitable for large-scale agricultural analytics.

Big Data Processing

Apache Spark can process large volumes of historical agricultural data using batch processing.

Apache Spark Streaming or Apache Flink can process real-time sensor data to detect immediate changes in soil and environmental conditions.

This provides both:

Real-Time Processing → Immediate Farm Decisions

Batch Processing → Historical Analysis and Prediction

Machine Learning Applications

Machine learning can be used for several important agricultural applications.

Crop Yield Prediction

Machine learning models can predict crop yield using **historical crop data, soil conditions, weather information, rainfall, and farming practices.**

Crop Disease Detection

Drone or smartphone images can be analyzed using **computer vision and deep learning models** to identify signs of crop diseases at an early stage.

Smart Irrigation

The system can determine when and how much water is required by analyzing **soil moisture, crop type, weather conditions, rainfall predictions, and temperature.**

For example, if soil moisture is already sufficient and rainfall is expected, the system can recommend delaying irrigation.

Pest Detection

Historical and real-time environmental data can be analyzed to identify conditions that may increase the possibility of pest attacks. Farmers can receive early warnings and take preventive actions.

Smart Farming Data Flow

Soil Sensors + Weather Stations + Drone Images + Satellite Data

↓

IoT Gateway

↓

Kafka / Cloud Data Ingestion

↓

Cloud Data Lake

↓

Spark / Flink Processing

↓

Machine Learning Models

↓

Crop Health + Yield Prediction + Disease Detection + Irrigation Recommendation

↓

Farmer Dashboard / Automated Irrigation System

Edge Computing

Agricultural areas may have poor or unreliable internet connectivity. Therefore, **edge computing** can be used to process important sensor data close to the farm.

For example, an edge device can immediately detect very low soil moisture and activate irrigation without waiting for a cloud response.

More complex analytics and historical processing can still be performed in the cloud.

Security and Reliability

IoT devices should use **secure authentication** to prevent unauthorized devices from connecting to the farming network.

Data should be encrypted during transmission and storage. **Role-Based Access Control** should restrict farm data to authorized users.

Device monitoring should detect unusual sensor behaviour or compromised devices. Regular backups, data replication, and disaster recovery mechanisms should be implemented to prevent data loss.

Scalability

The cloud architecture can automatically scale when more farms, sensors, drones, or data sources are added.

Distributed processing allows thousands or millions of sensor readings to be processed efficiently. Cloud storage can also expand as the amount of agricultural data increases.

Expected Benefits

The proposed smart farming solution provides:

- **Improved crop yield**
- **Reduced water consumption**
- **Early disease detection**
- **Efficient fertilizer and pesticide usage**
- **Reduced farming costs**
- **Real-time crop monitoring**
- **Better resource utilization**
- **Data-driven decision-making**

Conclusion

A combination of **IoT, Big Data, cloud computing, machine learning, edge computing, and real-time analytics** can transform traditional agriculture into smart and precision-based farming. The proposed architecture collects data from sensors, drones, satellites, and weather stations, processes it using technologies such as **Kafka, Spark, and Flink**, and provides useful predictions and recommendations to farmers.

The solution is **scalable, secure, reliable, and practical**, helping farmers improve productivity while reducing water, fertilizer, and other resource consumption.

Industry Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Context	20%	Demonstrates strong understanding of real-world problem context	Good understanding	Basic understanding	Weak understanding

Application of Big Data Concepts	25%	Applies highly relevant big data ideas and tools	Mostly appropriate application	Partial application	Weak application
Solution Design	25%	Solution is practical, scalable, and well-reasoned	Reasonable solution	Basic solution with gaps	Unclear solution
Security / Risk Awareness	15%	Strong awareness of security and operational risks	Reasonable awareness	Limited awareness	Poor awareness
Clarity and Professionalism	15%	Well-structured and professional response	Generally clear	Moderately clear	Poorly presented

Total Marks Awarded: